

ASSIGNMENT 1

Apache Hadoop MapReduce

Bandwidth Utilization Analysis Using Python mrjob

Student	Swasti M. Petkar
Semester	7th Semester – CSE (AI&ML)
Technology	Apache Hadoop MapReduce
Framework	Python mrjob
Input	Apache-style web server access log
Subject	Data Engineering and MLOps (BAD714C)
USN	4GW23CI055
Date of Submission	26/09/2025

This report documents the local and Hadoop/YARN execution of the MapReduce program, including HDFS storage, terminal execution evidence, output verification, and final results.

1. Objective

The objective of this assignment is to implement a Hadoop MapReduce solution using Python's mrjob framework to analyse raw web server access logs. The program extracts the client IP address and response bytes from each log record and calculates the total bandwidth transferred for every client IP address.

2. Problem Statement

A web hosting provider stores raw web server access logs. Each record contains a client IP address, date and time, HTTP request information, response status, and the number of response bytes transferred. The task is to read the access log, extract the client IP and response bytes, group records belonging to the same IP address, and calculate the sum of response bytes for each IP.

Total Bandwidth for an IP = Sum of response bytes for all requests from that IP.

To write a MapReduce program using **Python's mrjob** framework that processes an access log file, extracts client IP addresses and response bytes, and outputs the **total bandwidth (bytes transferred)** for each IP address.

3. Tools Used and Their Purpose

Tool / Technology	Purpose
Python 3.11.9	Programming language used for the MapReduce implementation and execution.
mrjob 0.7.4	Python framework used to write and run the MapReduce job.

Apache Hadoop 3.5.0	Distributed processing framework used for Hadoop execution.
HDFS	Distributed file system used to store the access log input.
YARN	Resource management and execution framework used to run the MapReduce job.
WSL2 / Ubuntu	Linux environment used to configure and run Hadoop locally.
Java 17	Runtime required by Apache Hadoop.

4. Input Dataset

The actual access log used for this implementation contained four Apache-style web server records. The last field in each record represents the response bytes.

```
192.168.1.10 - - [22/Sep/2026:10:00:01] "GET /index.html HTTP/1.1"
200 500
192.168.1.20 - - [22/Sep/2026:10:00:02] "GET /home.html HTTP/1.1"
200 1000
192.168.1.10 - - [22/Sep/2026:10:00:03] "GET /image.jpg HTTP/1.1"
200 2500
192.168.1.20 - - [22/Sep/2026:10:00:04] "GET /about.html HTTP/1.1"
200 700
```

5. MapReduce Working

Input: The access.log file is read one record at a time.

Mapper: Each line is split into fields. The first field is extracted as the client IP and the last field is converted to an integer representing response bytes. The mapper emits (IP, bytes).

Shuffle and Sort: Hadoop groups all byte values associated with the same IP address.

Reducer: The reducer receives an IP and all its byte values and calculates their sum.

Output: The final output contains each client IP address and its total transferred bandwidth.

6. Python MapReduce Program

File Edit View

```
from mrjob.job import MRJob

class BandwidthAnalysis(MRJob):

    def mapper(self, _, line):
        fields = line.split()

        ip = fields[0]
        bytes_transferred = int(fields[-1])

        yield ip, bytes_transferred

    def reducer(self, ip, bytes_values):
        total_bandwidth = sum(bytes_values)

        yield ip, total_bandwidth

if __name__ == '__main__':
    BandwidthAnalysis.run()
```

7. Execution

7.1 Local mrjob Execution

The program was first tested locally using Python 3.11.9 and mrjob 0.7.4. The command used was:

```
python bandwidth.py access.log
```

The local execution completed successfully and produced the two IP-level bandwidth totals.

```
Command Prompt
swasti_m_petkar@LAPTOP-IN

Installing collected packages: PyYAML, mrjob
Successfully installed PyYAML-6.0.3 mrjob-0.7.4

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ python --version
Python 3.11.9
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ pip show mrjob
Name: mrjob
Version: 0.7.4
Summary: Python MapReduce framework
Home-page: http://github.com/Yelp/mrjob
Author: David Marin
Author-email: dm@davidmarin.org
License: Apache
Location: /home/swasti_m_petkar/mrjob-env/lib/python3.11/site-packages
Requires: PyYAML
Required-by:
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ cd ~
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ ls -l bandwidth.py access.log
-rwxr-xr-x 1 swasti_m_petkar swasti_m_petkar 302 Sep 25 17:29 access.log
-rwxr-xr-x 1 swasti_m_petkar swasti_m_petkar 426 Sep 25 17:31 bandwidth.py
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ python bandwidth.py access.log
No configs found; falling back on auto-configuration
No configs specified for inline runner
Creating temp directory /tmp/bandwidth.swasti_m_petkar.20260925.174609.825978
Running step 1 of 1...
job output is in /tmp/bandwidth.swasti_m_petkar.20260925.174609.825978/output
Streaming final output from /tmp/bandwidth.swasti_m_petkar.20260925.174609.825978/output...
"192.168.1.20" 1700
```

Figure 1. Local mrjob execution showing Python 3.11.9, mrjob 0.7.4, and the final local output.

7.2 Hadoop and YARN Configuration

For Hadoop execution, Apache Hadoop 3.5.0 was configured with HDFS and YARN. The Hadoop services were started using the HDFS and YARN startup scripts, and the running Java processes were verified using jps.

```
start-dfs.sh
jps
start-yarn.sh
jps
```

```
2026-09-25 17:17:02,410 INFO common.Storage: setMostRecentCheckpointInfo txid is 0, time is 1790356622410
2026-09-25 17:17:02,430 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2026-09-25 17:17:02,441 INFO blockmanagement.DatanodeManager: Slow peers collection thread shutdown
2026-09-25 17:17:02,462 INFO namenode.FSNamesystem: Stopping services started for active state
2026-09-25 17:17:02,463 INFO namenode.FSNamesystem: Stopping services started for standby state
2026-09-25 17:17:02,469 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2026-09-25 17:17:02,471 INFO namenode.NameNode: SHUTDOWN_MSG:
/*****
SHUTDOWN_MSG: Shutting down NameNode at LAPTOP-IN2IMP01/127.0.1.1
*****/
swasti_m_petkar@LAPTOP-IN2IMP01:~/hadoop-3.5.0$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [LAPTOP-IN2IMP01]
LAPTOP-IN2IMP01: Warning: Permanently added 'laptop-in2imp01' (ED25519) to the list of known hosts.
swasti_m_petkar@LAPTOP-IN2IMP01:~/hadoop-3.5.0$ jps
2784 NameNode
3146 SecondaryNameNode
3309 Jps
2909 DataNode
swasti_m_petkar@LAPTOP-IN2IMP01:~/hadoop-3.5.0$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
swasti_m_petkar@LAPTOP-IN2IMP01:~/hadoop-3.5.0$ jps
3424 ResourceManager
2784 NameNode
3929 Jps
3562 NodeManager
3146 SecondaryNameNode
2909 DataNode
swasti_m_petkar@LAPTOP-IN2IMP01:~/hadoop-3.5.0$ |
```

Figure 2. Hadoop HDFS and YARN services verified using jps.

The verification showed the expected NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager processes.

7.3 HDFS Input Storage

The access log was copied into the Ubuntu environment and uploaded to the HDFS input directory. The HDFS file was then listed and displayed to verify the stored input.

```
hdfs dfs -mkdir -p /input
hdfs dfs -put ~/access.log /input/
hdfs dfs -ls /input
hdfs dfs -cat /input/access.log
```

```
Command Prompt
swasti_m_petkar@LAPTOP-IN

Peak Map Virtual memory (bytes)=2763018240
Peak Reduce Physical memory (bytes)=233381888
Peak Reduce Virtual memory (bytes)=2766303232
Physical memory (bytes) snapshot=824905728
Reduce input groups=2
Reduce input records=4
Reduce output records=2
Reduce shuffle bytes=98
Shuffled Maps =2
Spilled Records=8
Total committed heap usage (bytes)=407896064
Virtual memory (bytes) snapshot=8291426304

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

job output is in hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425/output
Streaming final output from hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425/output...
"192.168.1.10" 3000
"192.168.1.20" 1700
Removing HDFS temp directory hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425...
Removing temp directory /tmp/bandwidth.swasti_m_petkar.20260925.175710.858425...
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ hdfs dfs -ls /input
Found 1 items
-rw-r--r-- 1 swasti_m_petkar supergroup 302 2026-09-25 17:28 /input/access.log
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ hdfs dfs -cat /input/access.log
192.168.1.10 - - [22/Sep/2026:10:00:01] "GET /index.html HTTP/1.1" 200 500
192.168.1.20 - - [22/Sep/2026:10:00:02] "GET /home.html HTTP/1.1" 200 1000
192.168.1.10 - - [22/Sep/2026:10:00:03] "GET /image.jpg HTTP/1.1" 200 2500
192.168.1.20 - - [22/Sep/2026:10:00:04] "GET /about.html HTTP/1.1" 200 700(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ |
```

Figure 3. HDFS verification showing /input/access.log and the four input records.

8. Hadoop MapReduce Execution

The MapReduce job was executed using the Hadoop runner with the HDFS input file. Python 3.11.9 was explicitly supplied to the Hadoop task environment to ensure compatibility with mrjob 0.7.4.

```
python bandwidth.py -r hadoop --task-python-bin
/usr/local/bin/python3.11 hdfs:///input/access.log
```

The Hadoop job completed successfully. The execution statistics showed four map input records, two reduce input groups, and two reduce output records.

```

Command Prompt
swasti_m_petkar@LAPTOP-IN

Input split bytes=180
Map input records=4
Map output bytes=78
Map output materialized bytes=98
Map output records=4
Merged Map outputs=2
Peak Map Physical memory (bytes)=299495424
Peak Map Virtual memory (bytes)=2763018240
Peak Reduce Physical memory (bytes)=233381888
Peak Reduce Virtual memory (bytes)=2766303232
Physical memory (bytes) snapshot=824905728
Reduce input groups=2
Reduce input records=4
Reduce output records=2
Reduce shuffle bytes=98
Shuffled Maps =2
Spilled Records=8
Total committed heap usage (bytes)=407896064
Virtual memory (bytes) snapshot=8291426304
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
job output is in hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425/output
Streaming final output from hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425/output...
"192.168.1.10" 3000
"192.168.1.20" 1700
Removing HDFS temp directory hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425...
Removing temp directory /tmp/bandwidth.swasti_m_petkar.20260925.175710.858425...
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$

```

Figure 4. Successful Hadoop/YARN MapReduce execution and final output.

9. Output and Final Results

The mapper extracted the client IP and response bytes from each log record. The reducer then summed the bytes for each IP address.

Client IP Address	Calculation	Total Bandwidth
192.168.1.10	500 + 2500	3000 bytes
192.168.1.20	1000 + 700	1700 bytes
Overall	3000 + 1700	4700 bytes

Local vs Hadoop/YARN Verification

Execution Method	192.168.1.10	192.168.1.20
Local mrjob	3000 bytes	1700 bytes
Hadoop/YARN	3000 bytes	1700 bytes

Both execution methods produced identical results, verifying the correctness of the MapReduce implementation.

```
Command Prompt
swasti_m_petkar@LAPTOP-IN

Input split bytes=180
Map input records=4
Map output bytes=78
Map output materialized bytes=98
Map output records=4
Merged Map outputs=2
Peak Map Physical memory (bytes)=299495424
Peak Map Virtual memory (bytes)=2763018240
Peak Reduce Physical memory (bytes)=233381888
Peak Reduce Virtual memory (bytes)=2766303232
Physical memory (bytes) snapshot=824905728
Reduce input groups=2
Reduce input records=4
Reduce output records=2
Reduce shuffle bytes=98
Shuffled Maps =2
Spilled Records=8
Total committed heap usage (bytes)=407896064
Virtual memory (bytes) snapshot=8291426304

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

job output is in hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425/output
Streaming final output from hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425/output...
"192.168.1.10" 3000
"192.168.1.20" 1700
Removing HDFS temp directory hdfs:///user/swasti_m_petkar/tmp/mrjob/bandwidth.swasti_m_petkar.20260925.175710.858425...
Removing temp directory /tmp/bandwidth.swasti_m_petkar.20260925.175710.858425...
(mrjob-env) swasti_m_petkar@LAPTOP-IN2IMP01:~$ |
```

Figure 5. Final Hadoop MapReduce output used for result verification.

10. Conclusion

The Apache Hadoop MapReduce assignment was successfully implemented using Python and the mrjob framework. The web server access log was processed to extract client IP addresses and response bytes, and the MapReduce model was used to calculate the total bandwidth transferred by each IP address.

The input data was stored in HDFS and the MapReduce program was successfully executed through Hadoop/YARN. The local mrjob execution and the Hadoop/YARN execution produced the same final results: 192.168.1.10 → 3000 bytes and 192.168.1.20 → 1700 bytes. The overall bandwidth represented by the four records is 4700 bytes.

This implementation demonstrates the complete flow of a MapReduce application: input data storage in HDFS, mapping of log records into key-value pairs, shuffle and grouping by IP address, reduction by summing byte values, and generation of the final bandwidth utilization results.

Final Result

192.168.1.10	3000 bytes
192.168.1.20	1700 bytes
Total	4700 bytes